WEEK 05

Conversion and exceptions

This week: learn, practise, build

01 · Python

Conversion and exceptions
Understand the idea and trace examples.

02 · Practice

Practise the same Python concept.
Predict, change, diagnose and explain.

03 · Project

Proposal importer and audit
Build, discuss and test with a peer.

One weekly component moves the final project forward.

Python foundations

Conversion and exceptions
Concept → trace a small example → explain its behavior

External text, useful errors and an audit trail

Convert, then check range

"0" converts successfully.
A positive-count rule still rejects it.

An exception skips statements

Catch the expected ValueError.
A failed call did not return a count.

Keep the source evidence

One audit result per input.
Retain raw text and a useful reason.

Convert text, validate its value and preserve a useful failure reason.

Conversion can fail; valid numbers can be unsuitable

Convert a representation

int("4") produces integer 4. int("many") raises ValueError.

Check an allowed range

Zero converts successfully but can fail a positive-count rule. Conversion is not validation.

Handle a known failure

try runs the operation. except ValueError handles that failure; later input can still be checked.

Understand the behavior before changing the example.

Parsing and validation answer different questions

1

Raw text

"4", "0", "many"
Keep the original claim

2

Conversion

Can int() parse it?
Malformed text fails

3

Domain check

Is the count permitted?
Zero parses but fails here

Read the diagram, then trace the same steps in code.

Syntax to read and reuse

Python / syntax	Meaning	When to use it
csv.DictReader(stream)	Read rows as dictionaries	CSV fields arrive as text
int(raw)	Convert numeric text	May raise ValueError
raise ValueError(...)	Reject an invalid value	Make failure explicit
try / except ValueError	Handle a known failure	Record a useful reason

Python Basics · sections 09, 11, 16 · use only when you need a lookup.

Clean and convert one string

```
raw = " 3 "
cleaned = raw.strip()
count = int(cleaned)
print(repr(raw))
print(count, type(count).__name__)
```

1. Predict each printed result first.
2. Run this small example on its own.
3. Explain the operation before combining it.

Learn the single operation before the combined example.

Handle a conversion failure

```
raw = "many"
try:
    count = int(raw)
    print("Converted:", count)
except ValueError:
    print("Not an integer")
print("Continue")
```

1. Predict each printed result first.
2. Run this small example on its own.
3. Explain the operation before combining it.

Learn the single operation before the combined example.

Check a missing field before string methods

```
01 row = {"source_id": "A"}
02 raw = row.get("count_text")
03 if raw is None:
04     print("Missing count")
05 else:
06     print(raw.strip())
```

1. Predict each printed result first.
2. Run this small example on its own.
3. Explain the operation before combining it.

Learn the single operation before the combined example.

Text repetition versus numeric multiplication

```
raw = "4"
print(raw, type(raw).__name__)
count = int(raw)
print(count, type(count).__name__)
print(raw * 2)
print(count * 2)
```

1. Expected: 4 str; 4 int; 44; 8.
2. Conversion produces a new value.
3. Try "many": locate the failing statement.

Predict → execute → explain the value at each step.

CSV fields arrive as strings

```
01 import csv
02 from io import StringIO
03 text = "batch_id,reported_containers\nDEMO,4\n"
04 row = next(csv.DictReader(StringIO(text)))
05 print(row["reported_containers"])
06 print(type(row["reported_containers"]).__name__)
07 assert row["reported_containers"] == "4"
```

1. The field contains text "4".
2. Its Python type is str.
3. Reading and converting are separate steps.

Numeric-looking CSV fields are still strings.

int(): convert numeric text

```
raw = "4"
containers = int(raw)
planned = 3 * containers
print(containers, planned)
assert planned == 12
```

1. Convert the string "4" to integer 4.
2. Use the converted value in multiplication.
3. Try invalid text and read the exception.

int() creates a number that arithmetic can use.

Conversion errors and validation errors

```
01 def parse_containers(raw):
02     count = int(raw)
03     if count <= 0:
04         raise ValueError("containers must be positive")
05     return count
06
07 for raw in ["4", "0", "many"]:
08     try:
09         print(parse_containers(raw))
10     except ValueError as error:
11         print("Review:", str(error))
```

1. 4 passes conversion and range checks
2. 0 fails the positive-request rule
3. many fails inside int

Zero can parse successfully and still be rejected.

argparse: read a typed argument

```
01  import argparse
02  parser = argparse.ArgumentParser()
03  parser.add_argument("--containers", type=int, default=4)
04  args = parser.parse_args(["--containers", "2"])
05  print(args.containers)
06  assert args.containers == 2
```

1. parse_args() returns an argument namespace.
2. type=int converts the supplied text.
3. args.containers retrieves the parsed value.

Argument parsing and range validation are separate steps.

Hands-on practice

Week_05.ipynb · Session 2
Predict → run → change → explain

Your turn: practise the same Python skills

P1–P2 · Easy

Use the approach hints. Write the code yourself in a blank cell.

P3 · Core build

Implement a complete small program. Write expected results and assertions.

P4–P5 · Optional

Apply the skill to a new case, then attempt a harder challenge if time permits.

Write your own code, predict the result, then test and explain it.

Check your work before asking for help

A complete implementation

Write the whole operation. Do not stop at one changed character.

Independent expected results

Calculate a normal case and an edge case before running your program.

A peer can explain it

Swap driver and reviewer. Explain one line and one test.

Distinguish successful conversion, invalid text and an invalid range.

Project studio

Proposal importer and audit
Short design briefing → most time for your work

The people’s task this week

1

Someone needs help

Messages and table rows may contain missing or unclear counts.

2

This week, you will

Check each incoming row and keep a reason for its outcome.

3

A useful result

A human can follow up unclear information before it becomes an available offer.

Use the supplied starting files. Focus on one useful result and check it.

Keep source evidence through the pipeline

1

Read

Preserve source record ordinals and raw fields.

2

Normalize

Apply explicit transformations without inventing missing claims.

3

Decide

Validate individual rows and compare duplicate groups.

4

Record

Save a proposal or a reasoned hold/rejection.

A usable proposal and its audit answer different questions.

Your implementation TODO

Supplied starting point

The CSV fixture and audit helper are provided. The helper supplies parsing, grouping and audit assembly.

You implement / do

Implement the row validator: normalize the ID and description, convert both count fields, and raise ValueError for invalid rows.

Check the evidence

Six-row audit → 2 proposed, 2 held, 2 rejected. Test nonnumeric, zero and negative counts; keep the original source evidence.

Account for every source row; proposals do not change availability.

Show the component working

Step	What to show
1 · Locate	Identify the input, this component and its next user.
2 · Implement	Implement validate_offer_row; run supplied audit helper.
3 · Check	Six rows → 2 proposed, 2 held, 2 rejected.
4 · Explain	Show your code, its actual result and why it behaves that way.

Account for every source row; proposals do not change availability.

Further reading in English

1. ## Python CSV documentation

   https://docs.python.org/3/library/csv.html

2. ## Python errors and exceptions

   https://docs.python.org/3/tutorial/errors.html

Read one relevant section, then bring an example and a question.